

Modern Portfolio Theory: Efficient Frontier & Backtesting

Research question

Using real historical price data, what does the Markowitz efficient frontier look like for a diversified basket of 20 real US stocks, and how does the resulting optimal (max-Sharpe) portfolio actually perform out-of-sample compared to a simple S&P 500 benchmark?

This project uses **only real market data** — no simulated returns, no synthetic covariance matrices. All prices come from Yahoo Finance via the `yfinance` library.

Data source

- **Yahoo Finance** (via `yfinance`) – daily adjusted close prices for 20 large-cap US stocks across multiple sectors (see `data/tickers.csv`), plus `^GSPC` (S&P 500) as the benchmark.
- Free, no API key required. Data quality is generally reliable for large-cap US equities but is not a paid institutional-grade feed – note this as a limitation if you push the analysis further (e.g. dividends, corporate actions, survivorship bias since all 20 tickers are still listed today).

Methodology

1. Universe

20 large-cap stocks spanning 9 sectors (`data/tickers.csv`), chosen for sector diversity rather than performance – this is a design choice to worth stating explicitly, since cherry-picking winners would bias the "optimal" portfolio's apparent performance.

2. In-sample / out-of-sample split

- **Training window:** e.g. 2018-01-01 to 2022-12-31 — used to estimate expected returns and the covariance matrix, and to solve for the efficient frontier.
- **Test window:** e.g. 2023-01-01 to today — used purely to backtest the portfolios chosen from the training window. The optimizer never sees this data. This split matters: an efficient frontier fit and evaluated on the same data will look artificially good.

3. Optimization

- Daily log returns → annualized mean return vector and covariance matrix.
- Efficient frontier via constrained quadratic optimization (`scipy.optimize.minimize` , SLSQP): minimize portfolio variance for a grid of target returns, weights sum to 1, no short-selling (weights ≥ 0) as the base case.
- Two portfolios highlighted on the frontier:
 - **Minimum-variance portfolio**

- **Maximum-Sharpe-ratio portfolio** (using a risk-free rate proxy, e.g. 13-week T-bill yield)

4. Backtest

- Apply the training-window optimal weights to actual out-of-sample price data (buy-and-hold, no rebalancing, as the simplest honest baseline).
- Compare cumulative return, annualized volatility, and Sharpe ratio against the S&P 500 over the same out-of-sample window.

5. Limitations to report honestly

- Historical mean returns are a notoriously noisy estimate of *expected* future returns — the "optimal" portfolio is optimal only with respect to the training window's realized statistics, not a true forward-looking expectation. Say this plainly in the writeup rather than overselling the result.

- No transaction costs, taxes, or rebalancing are modeled.
- Survivorship bias: all 20 tickers are large, currently-listed companies that didn't go bankrupt or get delisted during the sample period.
- No short-selling constraint (weights ≥ 0) is a simplification; note this if you relax it later.

How to run

```
pip install -r requirements.txt
jupyter notebook analysis.ipynb
```

No API key needed — `yfinance` pulls data anonymously from Yahoo Finance.

If a ticker fails to download (rate limits, delisting, etc.), the notebook will flag it — rerun or drop that ticker.

Project structure

```
portfolio-optimization/
├── README.md
├── requirements.txt
```

```
|— data/
|   |— tickers.csv          # the 20-
stock + benchmark universe
|   |— analysis.ipynb      # full
pipeline: fetch → optimize → backtest
→ plot
```